Exact likelihoods for branching-process clone counts: from generating functions to the FFT to working code (Galton–Watson and Bellman–Harris)

BIO-SBI — companion to `note.pdf` (B-row identifiability)

2026-11-09

Abstract

This note collects, in one place and in the order you need them, the theory and the algorithms behind the exact likelihoods used in this project: `gwexact.py` and `profile.py` (analysis scripts, branch `worktree-b-row-theory`), the `exact_gw` inference method (branch `exact-gw`) and `bhexact.py` (branch `bh-exact`). The chain is: probability generating functions (pgfs) encode the law of the counts (§2); branching turns into composition of pgfs for Galton–Watson (§3) and into an integral equation, or for integer Gamma lifetimes a system of ODEs, for Bellman–Harris (§4); the values of a pgf at roots of unity give its coefficients through a discrete Fourier transform computed by the FFT, with aliasing, damping and round-off as the controlling issues (§5); derivatives go through the same pipeline in forward mode (§6); the likelihood is a sum of table look-ups (§7); MLE, Fisher information, profiles and NUTS sit on top (§8). §9 gives pseudo-code, costs, the validation checklist and the pitfalls we actually hit. Each section ends with references (list in §10).

Contents

1 Notation and the two models **2**

2 Probability generating functions **2**

3 Galton–Watson: pgf by composition **3**
3.1 The recursion . . . 3
3.2 Observation, founders and selection by substitution . . . 4

4 Bellman–Harris: integral equation and the method of stages **4**
4.1 General lifetimes: the Bellman–Harris equation . . . 4
4.2 Exponential lifetimes: an ODE . . . 4
4.3 Integer Gamma shape: phases . . . 5
4.4 Numerical integration . . . 5

5 From pgf values to probabilities: DFT, FFT, aliasing **5**
5.1 Inversion on roots of unity . . . 5
5.2 The FFT . . . 6
5.3 Choosing the grid: tails, damping, round-off . . . 6

6 Exact derivatives **6**

7 The likelihood **7**

8 Inference on top of the exact likelihood **8**
8.1 Parameterisation . . . 8
8.2 Maximum likelihood, profiles, Fisher . . . 8
8.3 Posterior sampling: NUTS . . . 8

9 Implementation **9**
9.1 Algorithms . . . 9
9.2 Cost and memory . . . 9
9.3 Validation checklist . . . 9
9.4 Pitfalls we hit . . . 10
9.5 Extensions and limits . . . 10

10 References **11**

1 Notation and the two models

Two types, A and B, marked by the variables x and y. A cell of type $k \in \{A, B\}$ ends its life with one *outcome* o from its row $\pi_k = (\pi_{k,o})_o$ and is replaced by $\xi_{k,o} = (a_{ko}, b_{ko})$ daughters (numbers of A and B). With the six outcomes of `two_type` (death, persist, switch, div_AA, div_AB, div_BB),

$$\begin{aligned} \text{A:} \quad & (a,b) \in \{(0,0),(1,0),(0,1),(2,0),(1,1),(0,2)\}, \\ \text{B:} \quad & (a,b) \in \{(0,0),(0,1),(1,0),(2,0),(1,1),(0,2)\} \end{aligned}$$

(B's "persist" leaves a B, its "switch" an A).

- **Galton–Watson (GW).** Synchronous generations: every living cell has exactly one event per generation. Data at generation n: $Z_n = (Z_{n,A}, Z_{n,B})$.
- **Bellman–Harris (BH).** Continuous time: each cell lives an independent lifetime $\tau \sim \text{Gamma}(m,\ \mu_k/m)$ (mean μ_k, coefficient of variation $1/\sqrt{m}$), then draws its outcome; daughters are born with age 0. The founder is born at $t = 0$ with age 0. Data at time t (hours): $Z(t)$ = numbers of A and B cells *alive* at t. In the repository $m = 8$ by default.

In both cases a clone starts from one A cell (other founder laws in §3.2), clones are independent, each organoid is measured once, and only *detected* clones ($Z \neq (0,0)$, with perfect capture) are recorded.

Ordering trap. The repository's B row is ordered (death, to_A, persist, …); the analysis scripts use (death, persist, to_A, …). `bhexact.from_repo` reorders. Getting this wrong silently swaps persist and switching.

2 Probability generating functions

Definition 2.1. For a random vector $K = (K_A, K_B)$ with values in $\mathbb{N}^2$, the pgf is

$$G_K(x,y) = \mathbb{E}\big[x^{K_A} y^{K_B}\big] = \sum_{a,b \geq 0} \mathbb{P}(K = (a,b))\, x^a y^b, \qquad |x|, |y| \leq 1.$$

The series converges absolutely on the closed bidisk $|x|, |y| \leq 1$ (coefficients are non-negative and sum to 1), so G_K may be evaluated at *complex* points of modulus ≤ 1; this is what the FFT step uses.

Proposition 2.2 (Basic properties). *1.* $G_K(1,1) = 1$, $|G_K(x,y)| \leq 1$ *on the bidisk, and* $G_K(0,0) = \mathbb{P}(K = 0)$.

2. *The coefficients are the law:* $\mathbb{P}(K=(a,b)) = \frac{1}{a!\,b!}\partial_x^a\partial_y^b G_K(0,0)$*; two vectors with the same pgf on any open set of the bidisk have the same law.*

3. *Moments at* $(1,1)$*:* $\mathbb{E}K_A = \partial_x G_K(1,1)$*,* $\mathbb{E}[K_A(K_A-1)] = \partial_x^2 G_K(1,1)$*,* $\mathbb{E}[K_A K_B] = \partial_x\partial_y G_K(1,1)$ *(when finite).*

4. *Marginals:* $G_{K_A}(x) = G_K(x,1)$*,* $G_{K_B}(y) = G_K(1,y)$*.*

5. *Real coefficients give conjugate symmetry:* $G_K(\bar{x},\bar{y}) = \overline{G_K(x,y)}$*.*

Proof. All follow from term-by-term manipulation of an absolutely convergent power series; uniqueness in (2) is the identity theorem for power series. □

Lemma 2.3 (Sums and random sums)**.** *(a) If* K *and* K' *are independent,* $G_{K+K'} = G_K G_{K'}$*. (b) Let* $N=(N_A,N_B)$ *be a random number of cells of each type, and let each type-*k *cell, independently, produce a vector distributed like* $Y^{(k)}$ *with pgf* H_k*. Then the total* S *has pgf*

$$G_S(x,y) = G_N\big(H_A(x,y),\, H_B(x,y)\big).$$

Proof. (a) $\mathbb{E}[x^{K_A+K'_A}y^{K_B+K'_B}] = \mathbb{E}[x^{K_A}y^{K_B}]\,\mathbb{E}[x^{K'_A}y^{K'_B}]$ by independence. (b) Conditional on $N=(i,j)$, S is a sum of i independent copies of $Y^{(A)}$ and j of $Y^{(B)}$, so by (a) $\mathbb{E}[x^{S_A}y^{S_B} \mid N=(i,j)] = H_A^i H_B^j$. Take expectations over N. □

Example 2.4 (Offspring pgfs)**.** Each outcome contributes its probability times $x^a y^b$:

$$f_A(x,y) = \pi_{A,\text{death}} + \pi_{A,\text{persist}}x + \pi_{A,\text{to_B}}y + \pi_{A,AA}x^2 + \pi_{A,AB}xy + \pi_{A,BB}y^2,$$
$$f_B(x,y) = \pi_{B,\text{death}} + \pi_{B,\text{persist}}y + \pi_{B,\text{to_A}}x + \pi_{B,AA}x^2 + \pi_{B,AB}xy + \pi_{B,BB}y^2.$$

Mean matrix $M_{kj} = \partial_j f_k(1,1)$; second factorial moments $\partial_x^2 f_k = 2\pi_{k,AA}$, $\partial_x\partial_y f_k = \pi_{k,AB}$, $\partial_y^2 f_k = 2\pi_{k,BB}$.

References. Feller Vol. I, Ch. XI [1] (one variable, very gentle); Harris Ch. I–II [2]; Athreya & Ney Ch. I and V [3]; Kimmel & Axelrod [4].

3 Galton–Watson: pgf by composition

3.1 The recursion

Write $F_n = (F_{n,A}, F_{n,B})$ with $F_{n,k}$ the pgf of Z_n started from one type-k cell, and $f=(f_A,f_B)$.

Theorem 3.1 (Composition)**.** $F_0(x,y)=(x,y)$ *and, for* $n\ge 1$*,*

$$F_n = f\circ F_{n-1} = F_{n-1}\circ f \qquad (\text{so } F_n = f^{\circ n}).$$

Proof. First-step decomposition: the founder's daughters Z_1 have pgf f_k; each daughter of type j independently starts a clone whose generation-$(n-1)$ content has pgf $F_{n-1,j}$. Lemma 2.3(b) with $N=Z_1$, $H=F_{n-1}$ gives $F_{n,k} = f_k\circ F_{n-1}$. *Last-step decomposition:* each cell of Z_{n-1} independently produces offspring with pgf f; Lemma 2.3(b) with $N=Z_{n-1}$, $H=f$ gives $F_{n,k} = F_{n-1,k}\circ f$. □

Corollary 3.2 (Degree bound)**.** *With at most two daughters per event,* $F_{n,k}$ *is a polynomial of total degree* $\le 2^n$*; in particular* $Z_{n,A}, Z_{n,B} \le 2^n$*.*

Evaluation at a point is cheap. To evaluate $F_{n,A}$ at a number (x,y), iterate

$$(u,v)\leftarrow(x,y); \qquad \text{repeat } n \text{ times: } (u,v)\leftarrow(f_A(u,v),\, f_B(u,v)); \qquad F_{n,A}(x,y) = u.$$

This is the *last-step* form ($F_n = F_{n-1}\circ f$ applied right to left) and costs $O(n)$ multiplications per point, never expanding the degree-2^n polynomial.

3.2 Observation, founders and selection by substitution

Lemma 3.3 (Thinning and mislabelling)**.** *If each endpoint cell of type k is independently captured with probability c_k and, when captured, labelled j with probability C_{kj}, then the pgf of the* measured *counts is $F_{n,A} \circ F_0^{\mathrm{obs}}$ with*

$$F_{0,k}^{\mathrm{obs}}(x, y) = 1 - c_k + c_k\,(C_{kA}x + C_{kB}y).$$

Proof. Lemma 2.3(b) with $N = Z_n$ and $H = F_0^{\mathrm{obs}}$, the pgf of what one endpoint cell contributes to the measurement. □

In the recursion this just changes the starting point: $(u, v) \leftarrow F_0^{\mathrm{obs}}(x, y)$ instead of (x, y). *Founders:* if a clone starts from S cells ($\mathbb{P}(S = s) = \varpi_s$) each of type k with probability φ_k, the clone pgf is $\sum_s \varpi_s\big(\varphi_A F_{n,A} + \varphi_B F_{n,B}\big)^s$ (Lemma 2.3). *Selection:* detected clones have law $\mathbb{P}(Z = z \mid Z \neq 0) = p_z/(1 - p_{00})$ for $z \neq 0$, $p_{00} = G(0, 0)$.

Detection probability without cancellation. When p_{00} is close to 1 (prior draws with high death), $1 - p_{00}$ computed by subtraction loses digits. Work with survival probabilities $s_{n,k} = 1 - F_{n,k}(0, 0)$ directly:

$$s_{n,k} = 1 - f_k(1 - s_{n-1,A},\, 1 - s_{n-1,B}) = \sum_o \pi_{k,o}\Big(1 - (1 - s_{n-1,A})^{a_{ko}}(1 - s_{n-1,B})^{b_{ko}}\Big),$$

expanding $1 - (1 - s)^2 = 2s - s^2$ etc., so no near-1 numbers are subtracted. (`detection_success` in the GW simulator and `detection_and_grad` in `exact_gw` do this; with capture c start from $s_{0,k} = c_k$.)

References. Composition: Harris Ch. I–II [2], Athreya & Ney Ch. I.2, V [3], Feller Vol. I Ch. XII (branching processes) [1]. Thinning: any text on pgfs, e.g. [1, Ch. XI].

4 Bellman–Harris: integral equation and the method of stages

4.1 General lifetimes: the Bellman–Harris equation

Let $F_k(x, y, t)$ be the pgf of $Z(t)$ for a clone started at time 0 by a newborn type-k cell, and G_k the lifetime distribution function.

Theorem 4.1 (Bellman–Harris integral equation)**.**

$$F_k(x, y, t) = s_k\,\big(1 - G_k(t)\big) + \int_0^t f_k\big(F_A(x, y, t - u),\, F_B(x, y, t - u)\big)\, dG_k(u), \qquad s_A = x,\ s_B = y.$$

Proof. Condition on the founder's lifetime τ. If $\tau > t$ (probability $1 - G_k(t)$) the clone at t is the founder alone, pgf s_k. If $\tau = u \le t$, the founder is replaced at u by newborn daughters, each starting an independent clone with $t - u$ to run; by Lemma 2.3(b) their total has pgf $f_k(F(t - u))$. Integrate over u. □

For each grid point this is a nonlinear Volterra equation in t. A product-integration (convolution quadrature) scheme on a time grid of M steps costs $O(M^2)$ per point; since G_k has no atom at 0 the scheme is explicit. This is the route for non-integer Gamma shapes (not implemented).

4.2 Exponential lifetimes: an ODE

If $\tau \sim \mathrm{Exp}(\lambda_k)$ (Markov branching), the backward Kolmogorov equation is

$$\partial_t F_k = \lambda_k\big(f_k(F_A, F_B) - F_k\big), \qquad F_k(x, y, 0) = s_k.$$

(Differentiate Theorem 4.1 with $dG_k(u) = \lambda_k e^{-\lambda_k u} du$, or argue on the first event in $[0, h]$.)

4.3 Integer Gamma shape: phases

Lemma 4.2 (Erlang = sum of exponentials)**.** *If $E_1, \dots, E_m$ are i.i.d.* Exp(λ)*, then $E_1 + \cdots + E_m \sim$* Gamma$(m, 1/\lambda)$*, with mean m/λ. So a* Gamma$(m, \mu_k/m)$ *lifetime is m consecutive exponential* phases *of rate $\lambda_k = m/\mu_k$.*

Introduce $G_{k,j}(x, y, t)$, the pgf at time t for a clone started by one type-k cell at the *start of phase j* ($j = 1, \dots, m$). Phase is not observed: a cell in any phase counts as one living cell.

Theorem 4.3 (Phase ODEs)**.**

$$\begin{aligned} \partial_t G_{k,j} &= \lambda_k \big(G_{k,j+1} - G_{k,j}\big), && j = 1, \dots, m-1, \\ \partial_t G_{k,m} &= \lambda_k \big(f_k(G_{A,1}, G_{B,1}) - G_{k,m}\big), && \\ G_{k,j}(x, y, 0) &= s_k, && F_k(x, y, t) = G_{k,1}(x, y, t). \end{aligned}$$

Proof. Backward argument on $[0, h]$. A cell in phase $j < m$ leaves its phase with probability $\lambda_k h + o(h)$ and then is a type-k cell at the start of phase $j+1$; otherwise it is unchanged: $G_{k,j}(t+h) = (1 - \lambda_k h) G_{k,j}(t) + \lambda_k h G_{k,j+1}(t) + o(h)$. In phase m leaving the phase ends the life, and the cell is replaced by newborn daughters (phase 1), whose clones combine by Lemma 2.3(b) into $f_k(G_{A,1}(t), G_{B,1}(t))$. At $t = 0$ the clone is the cell itself. Let $h \to 0$. □

Per grid point this is a $2m$-dimensional complex ODE (16 components for $m = 8$). Observation (Lemma 3.3) enters as the initial value $G_{k,j}(0) = F^{\text{obs}}_{0,k}$; selection and founders as in GW.

Remark 4.4 (Unbounded counts). Unlike GW (Corollary 3.2) the BH count has no finite bound: $F(\cdot, t)$ is a power series, not a polynomial. Every BH table is a truncation, controlled through marginal tails (§5.3).

4.4 Numerical integration

The system is non-stiff on the unit circle ($|G| \le 1$, rates $\lambda_k = m/\mu_k \approx 0.27$–$0.4\,\text{h}^{-1}$). Use a *fixed-step* explicit Runge–Kutta method: an adaptive step would make the computed likelihood a non-smooth function of θ (step choices change with θ), which breaks gradients and HMC. `bhexact.py` uses the 8th-order Dormand–Prince tableau (DOP853) with $h = 1\,\text{h}$: halving h changes probabilities by $\le 10^{-15}$, while classical RK4 needs $h \approx 0.06\,\text{h}$ for 10^{-10}. All horizons come from one trajectory (integrate to the first horizon, record, continue).

References. BH equation: Harris Ch. VI [2]; Athreya & Ney Ch. IV [3]; Kimmel & Axelrod [4]. Backward equations for Markov branching: Karlin & Taylor [15]. Method of stages, phase-type distributions: Neuts [16], Asmussen [17]; the same device in biology is the "linear chain trick" (MacDonald [18]). Runge–Kutta and DOP853: Hairer, Nørsett & Wanner [14].

5 From pgf values to probabilities: DFT, FFT, aliasing

5.1 Inversion on roots of unity

Let $\omega_L = e^{2\pi i/L}$.

Lemma 5.1 (Orthogonality)**.** $\sum_{j=0}^{L-1} \omega_L^{jm} = L$ *if* $m \equiv 0 \pmod L$ *and* 0 *otherwise.*

Proof. For $m \not\equiv 0$ it is a geometric sum $(1 - \omega_L^{Lm})/(1 - \omega_L^m) = 0$. □

Theorem 5.2 (DFT inversion with folding)**.** *Let $G(x, y) = \sum_{a,b} p_{ab} x^a y^b$ and $0 < r \le 1$. Evaluate on the $L_A \times L_B$ grid $g_{jl} = G(r\omega_{L_A}^j, r\omega_{L_B}^l)$. Then*

$$c_{ab} := \frac{1}{L_A L_B} \sum_{j,l} g_{jl}\, \omega_{L_A}^{-ja} \omega_{L_B}^{-lb} = \sum_{s,t \ge 0} p_{a+sL_A,\, b+tL_B}\; r^{\,a+sL_A+b+tL_B}, \qquad 0 \le a < L_A,\ 0 \le b < L_B.$$

Proof. Substitute the series, exchange sums (absolute convergence), apply Lemma 5.1 in each variable. □

Corollary 5.3. *(i)* Exact recovery: *if $p_{ab} = 0$ whenever $a \geq L_A$ or $b \geq L_B$, then $p_{ab} = c_{ab} r^{-(a+b)}$; with $r = 1$, $p_{ab} = c_{ab}$. For GW, $L_A, L_B \geq 2^n + 1$ is always exact (Corollary 3.2).* *(ii)* Aliasing error *otherwise:* $\left|c_{ab} r^{-(a+b)} - p_{ab}\right| \leq \max(r^{L_A}, r^{L_B})\, \mathbb{P}(Z_A \geq L_A \text{ or } Z_B \geq L_B)$.

The transform $g \mapsto c$ is the 2-D DFT. With numpy's sign convention (`fft` uses $e^{-2\pi i\, jk/L}$),

$$p = \mathrm{Re}\big(\texttt{fft2}(g)\big)/(L_A L_B) \qquad (\text{for } r = 1),$$

not `ifft2`, which returns p with indices reversed ($p_{-a,-b}$). The real part discards round-off; the exact c are real.

5.2 The FFT

A direct DFT of length L costs $O(L^2)$; the Cooley–Tukey FFT costs $O(L \log L)$ by splitting the sum over even and odd indices recursively. A 2-D DFT is 1-D DFTs along each axis: $O(L_A L_B \log(L_A L_B))$. FFT libraries are fastest for lengths with small prime factors (`scipy.fft.next_fast_len`).

5.3 Choosing the grid: tails, damping, round-off

Truncation at $r = 1$ (analysis scripts, `bhexact.py`). Pick L so the missed mass is negligible. The marginal pgfs $F(x, 1)$ and $F(1, y)$ give the marginal laws on a long 1-D grid ($N = 8192$) cheaply, and

$$\mathbb{P}(Z_A \geq L_A \text{ or } Z_B \geq L_B) \leq \mathbb{P}(Z_A \geq L_A) + \mathbb{P}(Z_B \geq L_B).$$

Examples: GW $n = 10$ at the reference truth, $L = 384$ reproduces the alias-free $L = 1025$ table to 10^{-17}; BH at 240 h, $(L_A, L_B) = (768, 512)$ misses $6 \cdot 10^{-9}$ (`bhexact_validation.md`).

Damping (`exact_gw`). Evaluating at radius $r < 1$ multiplies the folded terms by r^L, but computing $p = c\, r^{-(a+b)}$ multiplies the absolute round-off of c (about $\varepsilon \approx 10^{-16}$ times $\max|g| \leq 1$) by up to $r^{-(L_A+L_B)}$. Choosing r so that this amplification is at most D bounds

$$\text{round-off} \lesssim D\varepsilon, \qquad \text{aliasing} \lesssim \mathbb{P}(\text{count} \geq L) \cdot D^{-1/2} \text{ to } D^{-1}$$

(depending on how the budget is split between the axes). `exact_gw` uses $D = 10^4$ with $L =$ next fast length $\geq 1.25\times$(largest observed count)$+16$, i.e. a grid sized by the data rather than by the model's tail. Damping too hard is harmful: an early attempt with $r^L = 10^{-12}$ amplified round-off to 10^{24} and returned garbage.

Round-off floor. Even at $r = 1$ the computed p_{ab} carry absolute errors $\approx 10^{-17}$ and can be slightly negative. Probabilities below $\approx 10^{-15}$ are noise: floor them before taking logs (§7).

Symmetry. Real coefficients give $g(\bar{x}, \bar{y}) = \overline{g(x, y)}$ (Proposition 2.2), i.e. $g_{L_A - j,\, L_B - l} = \overline{g_{jl}}$: only half the grid needs the expensive evaluation (`bhexact.py` evaluates $j \leq L_A/2$ and fills the rest).

References. DFT and aliasing: Briggs & Henson [7]; FFT practice: Press et al. Ch. 12 [8]; Cooley & Tukey [9]. Inversion of pgfs specifically, including damping and error analysis: Abate & Whitt [10, 11].

6 Exact derivatives

Linearity. The map from grid values to coefficients is linear and does not depend on θ, so $\partial_\theta p = \mathrm{DFT}(\partial_\theta g)$: differentiate the pgf values, then apply the same FFT.

GW, forward mode. For a direction d in parameter space, differentiate the recursion $F_n = f(F_{n-1};\theta)$:

$$\dot F_{n,k} = \underbrace{(\partial_\theta f_k)[d](F_{n-1})}_{\text{explicit}} + \partial_x f_k(F_{n-1})\,\dot F_{n-1,A} + \partial_y f_k(F_{n-1})\,\dot F_{n-1,B}, \qquad \dot F_0 = 0.$$

For the direction "increase $\pi_{k,o}$" the explicit term is the monomial $x^{a_{ko}}y^{b_{ko}}$ evaluated at F_{n-1} in row k, zero in the other row. All 12 directions can be carried in one vectorised pass (`law_and_jac` in `profile.py`). Simplex constraints are handled afterwards by the chain rule: e.g. the direction $e_{k,o} - e_{k,\text{death}}$, or the softmax/stick-breaking Jacobian.

BH, tangent ODE. Differentiate the right-hand side $R(G;\theta)$ of Theorem 4.3: $\dot G' = J_R(G)\,\dot G + \partial_\theta R$, integrated with the *same* Runge–Kutta steps alongside G. This gives the exact derivative of the *discrete* solution (discretise-then-differentiate), which differs from the derivative of the true solution by the same $O(h^8)$ as the solution itself. For mean lifetimes, $\lambda_k = m/\mu_k$ gives $\partial\lambda_k/\partial\log\mu_k = -\lambda_k$, so $\partial_{\log\mu_k} R$ is minus the type-k rows of R.

Conditioning. For the detected law $\tilde p_z = p_z/(1-p_{00})$:

$$\partial\log\tilde p_z = \frac{\partial p_z}{p_z} + \frac{\partial p_{00}}{1-p_{00}} \qquad (z \neq 0).$$

Check. Always compare with central finite differences: GW $\sim 10^{-11}$ relative, BH $6\cdot10^{-12}$–$3\cdot10^{-10}$.

References. Forward-mode (tangent) differentiation: Griewank & Walther [12]; sensitivity equations for ODEs: Hairer, Nørsett & Wanner [14].

7 The likelihood

Proposition 7.1. *For a design with endpoints h and observed detected clones $z_{h,1},\dots,z_{h,N_h}$, independent across clones and endpoints,*

$$\ell(\theta) = \sum_h \sum_{i=1}^{N_h} \log\tilde p_h(z_{h,i};\theta) = \sum_h \sum_z n_h(z)\,\log\tilde p_h(z;\theta),$$

with $n_h(z)$ the number of clones with counts z at endpoint h.

One table per endpoint gives the probability of *every* count at once; the histogram form makes an evaluation one dot product, so the cost hardly depends on the number of organoids. For BH all endpoints come from one ODE trajectory.

Practicalities.

- **Floor:** use $\log\max(\tilde p, 10^{-15})$. Without it, a count with numerically zero probability under some θ gives $-\infty$ or, in the score, a ratio $\tilde p^\star/\tilde p$ of order 10^{280} (this happened in the profile optimisation).
- **Floor the detection probability** too ($1-p_{00} \geq 10^{-6}$ in `exact_gw`): where detection is nearly impossible the conditioned law is numerically meaningless and can produce spurious high likelihood.
- **Counts outside the table:** either size the grid from the data (`exact_gw`) or return $-\infty$ (`bhexact.log_lik`).

Derived quantities. Score $\nabla \ell = \sum n_h(z) \nabla \log \tilde{p}_h(z)$; per-clone Fisher information $I_h = \sum_z \tilde{p}_h(z)\, \nabla \log \tilde{p}_h(z) \nabla \log \tilde{p}_h(z)^\top$ (summing over the table, not the data); KL divergence between two parameter values $\sum_z \tilde{p}_h(z;\theta^\star) \log\big(\tilde{p}_h(z;\theta^\star)/\tilde{p}_h(z;\theta)\big)$, the expected per-clone log-likelihood ratio. These are what `note.pdf` uses.

References. Likelihood, score, Fisher information: Pawitan [19]; Lehmann & Casella Ch. 2 [20]; KL as expected log-likelihood ratio: Cover & Thomas Ch. 11 [21].

8 Inference on top of the exact likelihood

8.1 Parameterisation

Each probability row lives on a simplex. Two unconstrained parameterisations are used:

- **Softmax** (optimisation, `profile.py`): $\pi_k = \mathrm{softmax}(z_k)$, gradient $\partial\pi/\partial z = \mathrm{diag}(\pi) - \pi\pi^\top$.
- **Stick-breaking + logit** (Bayesian, `exact_gw`, same as the NPE prior): for a Dirichlet(α) row, $v_i \sim \mathrm{Beta}(\alpha_i, \sum_{j>i} \alpha_j)$ independently, $\pi_i = v_i \prod_{j<i}(1 - v_j)$; then $u_i = \mathrm{logit}\, v_i$. The log prior density in u is $\sum_i \big[\log \mathrm{Beta}(v_i) + \log v_i + \log(1 - v_i)\big]$ (Jacobian of the logit). Fixed entries (e.g. B divisions = 0) simply leave the row with fewer free entries.

8.2 Maximum likelihood, profiles, Fisher

L-BFGS(-B) with the exact gradient, several starts (prior draws), keep the best; profile likelihoods by fixing one function of θ (e.g. B-division mass δ) and optimising the rest. Fisher information at the optimum gives Wald intervals and Cramér–Rao bounds. At boundary truths (e.g. $\delta = 0$) the asymptotics are non-standard [22].

8.3 Posterior sampling: NUTS

Hamiltonian Monte Carlo uses $\nabla \log \pi(u \mid \text{data})$ to make long, informed proposals; the No-U-Turn Sampler (NUTS) chooses trajectory lengths automatically. `exact_gw`'s settings: 4 chains, 300 warm-up iterations (dual-averaging step size, dense mass matrix), 500 draws, maximum tree depth 8, target acceptance 0.8. Chains are started at the best of 8 L-BFGS optima from prior draws, keeping only optima within 20 nats of the best (a guard against poor local optima). Diagnostics: split-$\hat{R}$ and effective sample size. SBC means one full MCMC per calibration dataset, so it is run on few cases (configurable).

Cost. Each NUTS iteration needs up to 2^{depth} gradient evaluations. GW: one law plus 12 tangents at $n = 10$ is ~ 0.1–1 s, so a fit takes minutes. BH: one likelihood at 240 h is ~ 40 s on 2 threads (~ 4 s on 20) without gradients, so NUTS for BH needs a faster kernel (GPU, coarser h, smaller data-driven grids) before it is practical.

References. HMC: Neal [23]; Betancourt [24]; NUTS: Hoffman & Gelman [25]; $\hat{R}$ and ESS: Vehtari et al. [26]; SBC: Talts et al. [27]; L-BFGS: Nocedal & Wright Ch. 7 [13]; boundary asymptotics: Self & Liang [22].

9 Implementation

9.1 Algorithms

Algorithm 1 GW endpoint law and derivatives (`law_and_grads`, `law_and_jac`)

Require: rows π, generation n, grid sizes L_A, L_B (alias-free $\geq 2^n + 1$, or tail-checked), directions $d_1..d_D$

$x_j \leftarrow \omega^j_{L_A}$, $y_l \leftarrow \omega^l_{L_B}$ on the grid; $(U, V) \leftarrow F_0^{\text{obs}}(x, y)$; $\dot{U}_d, \dot{V}_d \leftarrow 0$

for $t = 1, \ldots, n$ **do**

  $\dot{U}_d \leftarrow (\partial_\theta f_A)[d](U, V) + \partial_x f_A\, \dot{U}_d + \partial_y f_A\, \dot{V}_d$; same for $\dot{V}_d$ with f_B ▷ use old U, V

  $(U, V) \leftarrow (f_A(U, V),\ f_B(U, V))$

end for

$p \leftarrow \mathrm{Re}\,\texttt{fft2}(U)/(L_A L_B)$; $\partial_d p \leftarrow \mathrm{Re}\,\texttt{fft2}(\dot{U}_d)/(L_A L_B)$

return p, ∂p ▷ founder mixture and conditioning applied afterwards

Algorithm 2 BH endpoint laws (`bhexact.law_bh`; `law_bh_jac` adds tangents)

Require: rows π, means μ_A, μ_B, integer shape m, horizons $t_1 < \cdots < t_H$, grid (L_A, L_B), step h

$\lambda_k \leftarrow m/\mu_k$; half grid $j \leq L_A/2$ (conjugate symmetry)

for all grid points (x, y) **in parallel do**

  $G_{k,j} \leftarrow s_k(x, y)$ for all k, j ▷ $s_A = x$, $s_B = y$, or F_0^{obs}

  for $i = 1..H$ **do**

    integrate Theorem 4.3 from t_{i-1} to t_i with fixed-step DOP853 ($\lceil (t_i - t_{i-1})/h \rceil$ steps)

    $F_i(x, y) \leftarrow G_{A,1}$

  end for

end for

fill the other half by $F(\bar{x}, \bar{y}) = \overline{F(x, y)}$; $p_i \leftarrow \mathrm{Re}\,\texttt{fft2}(F_i)/(L_A L_B)$

check tails with the 1-D marginals $F(x, 1)$, $F(1, y)$ on $N = 8192$ points

Algorithm 3 Exact log-likelihood (both models)

Require: data $\{z_{h,i}\}$, parameters θ

for each endpoint h **do**

  $n_h \leftarrow$ histogram of the observed (A, B) ▷ once per dataset

  $p_h \leftarrow$ endpoint law (Alg. 1 or 2), grid covering $\max z$

  $\tilde{p}_h \leftarrow p_h / \max(1 - p_{h,00},\ 10^{-6})$, $\tilde{p}_{h,00} \leftarrow 0$; $\tilde{p}_h \leftarrow \max(\tilde{p}_h, 10^{-15})$

end for

return $\sum_h \langle n_h, \log \tilde{p}_h \rangle$

9.2 Cost and memory

	GW, generation n	BH, horizon t, shape m
pgf evaluation	$O(nL_AL_B)$	$O(\frac{t}{h}\, s\, m\, L_AL_B)$, $s = 12$ stages (DOP853)
$+ D$ derivatives	$\times(1 + D)$	$\times(1 + D)$
FFT	$O(L_AL_B \log(L_AL_B))$ per table	same
memory	a few complex $L_A \times L_B$ arrays	$O(m)$ per point (streamed)
measured	$n = 10$, $L = 384$: ≈ 0.1–0.2 s	240 h, 768×512: 40 s (2 threads), ≈ 4 s (20)

9.3 Validation checklist

Every implementation in this project was checked against all of these; reuse them for new variants.

1. **Normalisation**: $\sum p = 1$ to 10^{-15}; negative mass $\lesssim 10^{-13}$.
2. **Means**: GW against $e_A M^n$ (agreement to machine precision); BH with $m = 1$ against the matrix-exponential means (`expected_counts`, $\leq 5 \cdot 10^{-12}$).
3. **Simulator agreement**: means (z-scores), single probabilities such as $P(1, 0)$, and the whole 2-D law by total variation with a *parametric-bootstrap* reference (multinomial samples from the exact law) and a pooled χ^2; repeat any isolated small p-value with fresh seeds before calling it a failure (one BH $p = 0.004$ was chance).
4. **Grid**: tail mass from the marginals; compare against a larger grid.
5. **Integrator** (BH): halve the step; changes must be at round-off level.
6. **Derivatives**: against central finite differences.
7. **End to end**: recover the truth on simulated data (MLE and posterior); for Bayesian runs, SBC on a few cases.

9.4 Pitfalls we hit

- **FFT sign convention**: `ifft2` instead of `fft2`$/L^2$ returns the table with reversed indices. The normalisation check passes anyway; the mean check catches it.
- **Row ordering**: the repository's B row is (death, to_A, persist); scripts use (death, persist, to_A).
- **Unnormalised rows**: posterior means read from files may not sum exactly to 1; renormalise each row before evaluating (a 1% deficit produced a fake heavy tail).
- **Over-damping**: amplification r^{-L} must stay modest ($\leq 10^4$).
- **Log of zero / exploding scores**: floor probabilities and detection.
- **Adaptive ODE steps**: make the likelihood non-smooth in θ; use fixed steps.
- **Low-order integrators**: RK4 needs $\sim 16\times$ more steps than DOP853 for 10^{-10} accuracy here.
- **CPU sharing**: the BH kernel and NUTS chains are parallel; cap threads when other jobs run.

9.5 Extensions and limits

- **Non-integer Gamma shape**: solve Theorem 4.1 by product integration ($O(M^2)$ per point), or approximate the lifetime by a phase-type distribution with integer phases.
- **Third type C**: a third variable z, 3-D grid and 3-D FFT; memory grows as L^3.
- **Quiescence, desynchronised starts, random founder age**: extra phases or a modified initial condition.
- **Repeated measurements of the same clone**: needs the joint pgf over times (multi-time generating functions); not covered.
- **Large clones**: memory $\propto L_A L_B$; damping or data-sized grids help, beyond that a saddle-point or moment-based approximation is needed.

Where the code is. `scripts/b_row_theory/gwexact.py`, `profile.py` (branch `worktree-b-row-theory`); `src/bio_sbi/inference/gw_exact.py`, `exact_gw.py`, `nuts.py`, `conf/inference/exact_gw.yaml` (branch `exact-gw`); `scripts/b_row_theory/bhexact.py`, `bhexact_validate.py`, `bhfisher.py`, `bhexact_validation.md` (branch `bh-exact`).

10 References

Chapter numbers refer to the editions listed and were given from memory; check the table of contents if a pointer seems off.

[1] W. Feller, *An Introduction to Probability Theory and Its Applications*, Vol. I, 3rd ed., Wiley, 1968. Ch. XI generating functions; Ch. XII compound distributions and branching processes.

[2] T. E. Harris, *The Theory of Branching Processes*, Springer, 1963. Ch. I single type, II multitype, VI age-dependent (Bellman–Harris) processes.

[3] K. B. Athreya and P. E. Ney, *Branching Processes*, Springer, 1972 (Dover 2004). Ch. I Galton–Watson, IV age-dependent, V multitype.

[4] M. Kimmel and D. E. Axelrod, *Branching Processes in Biology*, 2nd ed., Springer, 2015.

[5] P. Haccou, P. Jagers and V. A. Vatutin, *Branching Processes: Variation, Growth, and Extinction of Populations*, Cambridge University Press, 2005.

[6] P. Guttorp, *Statistical Inference for Branching Processes*, Wiley, 1991.

[7] W. L. Briggs and V. E. Henson, *The DFT: An Owner's Manual for the Discrete Fourier Transform*, SIAM, 1995.

[8] W. H. Press, S. A. Teukolsky, W. T. Vetterling and B. P. Flannery, *Numerical Recipes*, 3rd ed., Cambridge University Press, 2007, Ch. 12.

[9] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series", *Math. Comp.* 19 (1965) 297–301.

[10] J. Abate and W. Whitt, "Numerical inversion of probability generating functions", *Oper. Res. Lett.* 12 (1992) 245–251.

[11] J. Abate and W. Whitt, "The Fourier-series method for inverting transforms of probability distributions", *Queueing Systems* 10 (1992) 5–88.

[12] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed., SIAM, 2008.

[13] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006, Ch. 7.

[14] E. Hairer, S. P. Nørsett and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed., Springer, 1993 (Runge–Kutta methods, Dormand–Prince, DOP853).

[15] S. Karlin and H. M. Taylor, *A First Course in Stochastic Processes*, 2nd ed., Academic Press, 1975 (Markov branching processes, backward equations).

[16] M. F. Neuts, *Matrix-Geometric Solutions in Stochastic Models*, Johns Hopkins University Press, 1981 (phase-type distributions).

[17] S. Asmussen, *Applied Probability and Queues*, 2nd ed., Springer, 2003 (phase-type distributions).

[18] N. MacDonald, *Time Lags in Biological Models*, Lecture Notes in Biomathematics 27, Springer, 1978 (the linear chain trick).

[19] Y. Pawitan, *In All Likelihood*, Oxford University Press, 2001.

[20] E. L. Lehmann and G. Casella, *Theory of Point Estimation*, 2nd ed., Springer, 1998.

[21] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006.

[22] S. G. Self and K.-Y. Liang, "Asymptotic properties of maximum likelihood estimators and likelihood ratio tests under nonstandard conditions", *J. Amer. Statist. Assoc.* 82 (1987) 605–610.

[23] R. M. Neal, "MCMC using Hamiltonian dynamics", in *Handbook of Markov Chain Monte Carlo*, Chapman & Hall/CRC, 2011.

[24] M. Betancourt, "A conceptual introduction to Hamiltonian Monte Carlo", arXiv:1701.02434, 2017.

[25] M. D. Hoffman and A. Gelman, "The No-U-Turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo", *J. Mach. Learn. Res.* 15 (2014) 1593–1623.

[26] A. Vehtari, A. Gelman, D. Simpson, B. Carpenter and P.-C. Bürkner, "Rank-normalization, folding, and localization: an improved $\hat{R}$ for assessing convergence of MCMC", *Bayesian Analysis* 16 (2021) 667–718.

[27] S. Talts, M. Betancourt, D. Simpson, A. Vehtari and A. Gelman, "Validating Bayesian inference algorithms with simulation-based calibration", arXiv:1804.06788, 2018.